Acceso a ficheros

Contenido

1. Tipos de ficheros 3
1.1. Propiedades del Sistema 4
2. Ficheros de texto 4
2.1. Ficheros de texto plano 4
Escribir ficheros de texto plano 5
Leer ficheros de texto plano 6
2.2. Ficheros de configuración 7
Escribir ficheros de configuración 7
Leer ficheros de configuración 8
2.3. Ficheros XML 9
Escribir ficheros XML 11
Leer ficheros XML 12
3. Ficheros binarios 13
3.1. Serialización de objetos 14
Serializar un objeto 14
Deserializar un objeto 15
4. Estructuras de datos 16
4.1. ArrayList 16
Operaciones más comunes 16
4.2. HashMap 18
Operaciones más comunes 18
4.3. TreeSet 20
Operaciones más comunes 20
4.4. TreeMap 22
Operaciones más comunes 22
4.5. LinkedList 24

Operaciones más comunes 24
4.6. Comparativa de la JFC (Java Collection Framework) 27
5. Patrón Modelo-Vista-Controlador 28
5.1. Ejemplo de aplicación MVC 28
6. Ejercicios 44
7. Práctica 1 46
8. Práctica 2 48

1. Tipos de ficheros

Desde el punto de vista de un programador solamente distinguiremos entre dos tipos de ficheros:

- **Ficheros de texto** cuando el contenido del fichero contenga exclusivamente caracteres de texto (podemos leerlo con un simple editor de texto)

Extensión	Tipo de fichero
.txt	Fichero de texto plano
.xml	Fichero XML
.json	Fichero de intercambio de información
.props	Fichero de propiedades
.conf	Fichero de configuración
.sql	Script SQL
.srt	Fichero de subtítulo

- **Ficheros binarios** cuando no estén compuestos exclusivamente de texto. Pueden contener imágenes, videos, ficheros, . . . aunque también podemos considerar un fichero binario a un fichero de Microsoft Word en el que sólo hayamos escrito algún texto puesto que, al almacenarse el fichero, el procesador de texto incluye alguna información binaria

Extensión	Tipo de fichero
.pdf	Fichero PDF
.jpg	Fichero de imagen
.doc, .docx	Fichero de Microsoft Word
.avi	Fichero de video
.ppt, .pptx	Fichero de PowerPoint

A veces, en ficheros binarios, podremos encontrarnos con las extensiones *.bin* o *.dat* para hacer referencia a ficheros que contienen información binaria en un formato que no está ampliamente difundido. Serán simplemente ficheros que una aplicación determinada es capaz de leer/escribir de una forma específica solo definida para dicha aplicación.

1.1. Propiedades del Sistema

En cualquier caso, para el acceso a ficheros independientemente del tipo de los mismos, conviene conocer el funcionamiento de las System Properties de las que dispone Java en su API. Éstas permiten acceder a propiedades de la configuración del sistema y, entre otras, podemos encontrarnos con algunas muy interesantes relacionadas con este tema:

- "`file.separator`" Obtiene el caracter, según el S.Operativo, para la separación de las rutas (`/` ó `\`). También se puede utilizar la constante `File.separator`
- "`user.home`" Obtiene la ruta de la carpeta personal del usuario (que dependerá del S.Operativo en casa caso)
- "`user.dir`" Obtiene la ruta en la que se encuentra actualmente el usuario
- "`line.separator`" Obtiene el caracter que separa las líneas de un fichero de texto (difiere entre Windows/Linux)

En el caso de que se quiera acceder al valor de alguna de estas propiedades debe hacerse utilizando la llamada al método `System.getProperty(String)`

```
System.out.println("La carpeta de mi usuario es " +
System.getProperty("user.home"));
```

2. Ficheros de texto

En esta parte vamos a trabajar con 3 tipos de ficheros de texto:

- **Ficheros de texto plano** que contendrán texto *libre* y donde podremos escribir sin respetar ningún tipo de formato
- **Ficheros de configuración** que contendrán información de configuración para una aplicación. Tienen un formato específico y Java además proporciona una API para trabajar más cómodamente con ellos
- **Ficheros XML** que contienen información y acompañada de etiquetas que le dan significado. Tienen unas reglas y formato más o menos definido y Java proporciona una API para trabajar con ellos

Existen más tipos de ficheros de texto también muy extendidos como *.json* y *.csv* pero no serán estudiados en esta parte.

2.1. Ficheros de texto plano

Los ficheros de texto son aquellos que únicamente contienen texto, por lo que pueden ser editados directamente con cualquier editor de texto plano (Bloc de Notas, notepad++, . . .). Se podría decir que son aquellos ficheros que podrían ser *leídos* por cualquier persona. Son aquellos que normalmente se almacenan con la extensión *.txt* pero también podríamos incluir los scripts SQL (*.sql*), ficheros de código Java (*.java*), ficheros de configuración (*.ini*, *.props*, *.conf*, . . .), . . .

También se incluyen en la categoría de ficheros de texto los que además incluyen información adicional (siempre en forma de texto) que permiten interpretar los datos del

fichero de una manera u otra, añadiendo más información al mismo. Estos formatos son HTML, XML, JSON, . . .

En cualquier caso, desde Java siempre se podrán leer/escribir de la misma manera, según veremos a continuación. También veremos como pueden leerse/escribirse utilizando librerías aquellos ficheros de texto plano que contienen formato, como hemos comentado anteriormente, haciendo de esa forma mucho más fácil el trabajo.

Escribir ficheros de texto plano

```
FileWriter fichero = null;

PrintWriter escritor = null;



try {

  fichero = new FileWriter("archivo.txt");

  escritor = new PrintWriter(fichero);

  escritor.println("Esto es una linea del fichero");

} catch (IOException ioe) {

  ioe.printStackTrace();

} finally {

if (fichero != null)

  try {

    fichero.close();

  } catch (IOException ioe) { . . . }
```

```
}
```

Leer ficheros de texto plano

```
File fichero = null;

FileReader lector = null;

BufferedReader buffer = null;



try {

  buffer = new BufferedReader(new FileReader(new File("archivo.txt")));

  String linea = null;

  while ((linea = buffer.readLine()) != null)

    System.out.println(linea);

} catch (FileNotFoundException fnfe) {

  fnfe.printStackTrace();

} catch (IOException ioe) {

  ioe.printStackTrace();

} finally {

  if (buffer != null)

  try {

    buffer.close();

  } catch (IOException ioe) { . . . }

}
```

2.2. Ficheros de configuración

En la API de Java se incluyen librerías para trabajar con los ficheros de configuración. Puesto que todos siguen un mismo patrón, es la librería la que se encarga de acceder al fichero a bajo nivel y el programador sólo tiene que indicar a que propiedad quiere acceder o que propiedad quiere modificar, sin tener que añadir nada de código para leer o escribir el fichero tal y como hemos visto en el punto anterior.

Un ejemplo de fichero de configuración de esta librería de java sería el fichero que sigue:

configuracion.props

```
# Fichero de configuracion

# Thu Nov 14 10:49:39 CET 2013



user=usuario

password=micontrasena

server=localhost

port=3306
```

Escribir ficheros de configuración

Así, si queremos generar, desde Java, un fichero de configuración como el anterior:

```
. . .

Properties configuracion = new Properties();

configuracion.setProperty("user", miUsuario);

configuracion.setProperty("password", miContrasena);

configuracion.setProperty("server", elServidor);

configuracion.setProperty("port", elPuerto);

try {
```

```
  configuracion.store(new FileOutputStream("configuracion.props"),
                                          "Fichero de configuracion");
} catch (FileNotFoundException fnfe ) {
  fnfe.printStackTrace();
} catch (IOException ioe) {
  ioe.printStackTrace();
}
. . .
```

Leer ficheros de configuración

A la hora de leerlo, en vez de tener que recorrer todo el fichero como suele ocurrir con los ficheros de texto, simplemente tendremos que cargarlo e indicar de qué propiedad queremos obtener su valor (`getProperty(String)`).

```
. . .
Properties configuracion = new Properties();
try {
  configuracion.load(new FileInputStream("configuracion.props"));
  usuario = configuracion.getProperty("user");
  password = configuracion.getProperty("password");
  servidor = configuracion.getProperty("server");
  puerto = Integer.valueOf(configuration.getProperty("port"));
} catch (FileNotFoundException fnfe ) {
  fnfe.printStackTrace();
} catch (IOException ioe) {
```

```
  ioe.printStackTrace();

}

. . .
```

Para ambos casos, escribir y leer este tipo de ficheros, hay que tener en cuenta que, al tratarse de ficheros de texto, toda la información se almacena como si de un `String` se tratara. Por tanto, todos aquellos tipos `Date`, `boolean` o incluso cualquier tipo numérico serán almacenados en formato texto. Así, habrá que tener en cuenta las siguientes consideraciones:

- Para el caso de las fechas, deberán ser convertidas a texto cando se quieran escribir y nuevamente reconvertidas a `Date` cuando se lea el fichero y queramos trabajar con ellas
- Para el caso de los tipos `boolean`, podemos usar el método `String.valueOf(boolean)` para pasarlos a `String` cuando queramos escribirlos. En caso de que queramos leer el fichero y pasar el valor a tipo `boolean` podremos usar el método `Boolean.parseBoolean(String)`
- Para el caso de los tipos numéricos (`integer`, `float`, `double`) es muy sencillo ya que Java los convertirá a `String` cuando sea necesario al escribir el fichero. En el caso de que queramos leerlo y convertirlos a su tipo concreto, podremos usar los métodos `Integer.parseInt(String)`, `Float.parseFloat(String)` y `Double.parseDouble()`, según proceda

2.3. Ficheros XML

Los ficheros XML permiten el intercambio de información entre aplicaciones utilizando para ello un fichero de texto plano al que se le pueden añadir etiquetas para darle significado a cada uno de los valores que se almacenan.

Por ejemplo, el siguiente fichero XML podría ser el resultado de volcar una Base de Datos sobre productos de una compañia, de forma que dicha información podría ahora leerse desde otra aplicación e incorporarla. Se puede ver como a parte de los datos de dichos productos (sus nombres, precios, . . .) aparece otra información en forma de etiquetas (entre los caracteres < y >) que permite dar significado a cada dato almacenado en el fichero. Así, podemos saber de qué estamos hablando y a qué corresponde cada valor.

Obviamente, tal y como ocurría con los ficheros de configuración, toda la información se tiene que pasar a texto para poder crear el fichero y reconvertida a su tipo original cuando se cargue de nuevo. Las mismas consideraciones que hemos tenido en cuenta antes nos servirán ahora (ver aqui)

productos.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="no">
```

```xml
<xml>
<productos>
  <producto>
    <nombre>Cereales</nombre>
    <precio>3.45</precio>
  </producto>
  <producto>
    <nombre>Colacao</nombre>
    <precio>1.45</precio>
  </producto>
  <producto>
    <nombre>Agua mineral</nombre>
    <precio>1.00</precio>
  </producto>
</productos>
</xml>
```

La clase Java que definiría cada uno de los objetos que se representan por el fichero XML anterior, sería la siguiente:

Producto.java

```java
public class Producto {
  private String nombre;
  private float precio;
```

```
    public Producto(String nombre, float precio) {
      this.nombre = nombre;
      this.precio = precio;
    }

    public String getNombre () { return nombre; }
    public void setNombre (String nombre) { this.nombre = nombre; }

    public float getPrecio () { return precio; }
    public void setPrecio (float precio) { this.precio = precio; }
  }
```

Escribir ficheros XML

Si ahora queremos generar el fichero XML con toda la información de una colección de objetos *Producto*, podemos utilizar el siguiente código:

```
. . .
documento = dom.createDocument(null, "xml", null);
Element raiz = document.createElement("productos");
documento.getDocumentElement().appendChild(raiz);
Element nodoProducto = null , nodoDatos = null ;
Text texto = null;
```

```
for (Producto producto : listaProductos) {

  nodoProducto = documento.createElement("producto");

  raiz.appendChild(nodoProducto);

  nodoDatos = documento.createElement("nombre");

  nodoProducto.appendChild(nodoDatos);

  texto = documento.createTextNode(producto.getNombre());

  nodoDatos.appendChild(texto);

  nodoDatos = documento.createElement("precio");

  nodoProducto.appendChild(nodoDatos);

  texto = documento.createTextNode(producto.getPrecio());

  nodoDatos.appendChild(texto);

}

. . .
```

Leer ficheros XML

Y si lo que queremos es leer un fichero XML y cargarlo como una colección Java de objetos *Producto*, el ejemplo siguiente muestra un breve ejemplo acerca de cómo acceder a la información de dicho fichero XML.

```
. . .

NodeList productos = documento.getElementyByTagName("producto");

for (int i = 0; i < productos.getLength(); i++) {

  Node producto = productos . item ( i ) ;

  Element elemento = ( Element ) producto ;

  System.out.println(elemento.getElementsByTagName("nombre").item(0)
```

```
                    .getChildNodes().item(0).getNodeValue());
  System.out.println(elemento.getElementsByTagName("precio").item(0)
                    .gethildNodes().item(0).getNodeValue());
}
. . .
```

3. Ficheros binarios

Producto.java

```
public class Producto implements Serializable {
  private static final long serialVersionUID = 1L;
  private String nombre;
  private float precio;

  public Producto(String nombre, float precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  public String getNombre() { return nombre; }
  public void setNombre(String nombre) { this.nombre = nombre; }

  public float getPrecio() { return precio ; }
```

```
    public void setPrecio(float precio) { this.precio = precio; }

}
```

3.1. Serialización de objetos

Serializar es el proceso por el cual un objeto en memoria pasa a transformarse en una estructura que pueda ser almacenada en un fichero (persistencia). Al proceso contrario le llamaremos *deserializar*.

Hay que tener en cuenta que, durante el proceso de serialización, cada objeto se serializa a un fichero, por lo que si queremos almacenar todos los objetos de una aplicación, la idea más conveniente es tener todos éstos en alguna estructura compleja (y por comodidad dinámica) de forma que sea esta estructura la que serialicemos o deserialicemos para guardar o cargar los objetos de una aplicación. Estructuras como `ArrayList`, `Set` ó `HashMap` son clases muy utilizadas para trabajar de esta manera.

Serializar un objeto

```
. . .

ObjectOutputStream serializador = null;

try {

  serializador = new ObjectOutputStream(new
FileOutputStream("archivo.dat"));

  serializador.writeObject(listaProductos);

} catch (IOException ioe) {

  . . .

} finally {

  if (serializador != null)

    try {

      serializador.close();
```

```
    } catch (IOException ioe) {

      ioe.printStackTrace();

    }

  }

. . .
```

Deserializar un objeto

```
. . .

List<Producto> listaProductos = null;

ObjectInputStream deserializador = null;

try {

  deserializador = new ObjectInputStream(new
FileInputStream("archivo.dat"));

  listaProductos = (ArrayList<Producto>)deserializador.readObject();

} catch (FileNotFoundException fnfe ) {

  fnfe.printStackTrace();

} catch (ClassNotFoundException cnfe ) {

  cnfe.printStackTrace();

} catch (IOException ioe) {

  ioe.printStackTrace();

} finally {

  if (deserializador != null)

  try {
```

```
    deserializador.close();
  } catch (IOException ioe) {
    ioe.printStackTrace();
  }
}
. . .
```

4. Estructuras de datos

4.1. ArrayList

ArrayList es un *array* redimensionable que implementa el *interface* List. Permite todos los elementos, incluído el valor `null`. Como es redimensionable, dispone de método para modificar el tamaño del mismo.

Para el acceso concurrente con varios hilos, hay que tener en cuenta que esta estructura de datos no está sincronizada y su acceso deberá estarlo de forma externa. También se podría emplear la clase Vector que es igual que ésta pero está ya sincronizada.

Operaciones más comunes

Añadir un elemento

```
List<String> listaCadenas = new ArrayList<>();

String nuevaCadena = "Esto es una cadena";



listaCadenas.add(nuevaCadena);
```

Añadir un elemento en una posición determinada

```
int posicion = 3;
```

```
listaCadenas.add(3, nuevaCadena);
```

Reemplazar el elemento que ocupa una posición determinada

```
int posicion = 3;

String otraCadena = "Esto es otra cadena";

listaCadenas.set(3, otraCadena);
```

Eliminar un elemento

```
int posicion = 3;

listaCadenas.remove(posicion);
```

Obtener la referencia a un elemento

```
int posicion = 3;

String unaCadena = listaCadenas.get(posicion);
```

Conocer el número de elementos

```
System.out.println("El ArrayList tiene " + listaCadenas.size() + "
elementos");
```

Eliminar todos los elementos de la lista

```
listaCadenas.clear();
```

Añadir todos los elementos de otra lista

```
List<String> otraListaCadenas = new ArrayList<>();

. . .

. . .
```

```
// Se puede pasar como parámetro un objeto que implemente el interfaz Collection

listaCadenas.addAll(otraListaCadenas);
```

Comprobar si esta vacía

```
if (listaCadenas.isEmpty()) {

  System.out.println("La lista no tiene elementos");

}
```

Obtener un array (estático) con todos los elementos de la lista

```
String[] arrayCadenas = listaCadenas.toArray();
```

4.2. HashMap

HashMap es una tabla *hash* que implementa el *interface* Map. Permite el valor `null` tanto para valores como para claves. Al contrario que TreeSet no garantiza el orden de los elementos o que éste se mantenga.

Hay que tener en cuenta que el acceso a esta implementación no está sincronizado, pero existe una implementación similar y sincronizada, que es la clase Hashtable.

Operaciones más comunes

Añadir un elemento (pareja clave-valor)

```
Map<String, Libro> libros = new HashMap<,>();

. . .

Libro libro = new Libro();

libro.setTitulo("Secuestrado");

libro.setAutor("Robert Louis Stevenson");

. . .
```

```
libros.add(libro.getTitulo(), libro);
```

Obtener un elemento

```
String tituloLibro = "Secuestrado";

Libro esteLibro = libros.get(titulo);
```

Comprobar si existe una clave

```
String titulo = "Secuestrado";

if (libros.containsKey(titulo)) {

  System.out.println("El libro con el título " + titulo + " existe en tu
colección");

}
```

Comprobar si está vacío

```
if (libros.isEmpty()) {

  System.out.println("Tu colección de libros está vacía");

}
```

Eliminar todos los elementos

```
libros.clear();
```

Eliminar un elemento (por clave)

```
String titulo = "Secuestrado";

libros.remove(titulo);
```

Obtener una `Collection` con todos los valores

```
Collection<Libro> coleccionLibros = libros.values();
```

```
// ArrayList, TreeSet y LinkedList implementan el interface Collection
```

Obtener un `Set` con todas las claves

```
Set<String> titulos = libros.keySet();

// TreeSet y HashSet implementan el interface Set
```

Comprobar el tamaño del `Map`

```
System.out.println("Tienes " + libros.size() + " libros en tu
colección");
```

Concatenar todos los elementos de otro `Map`

```
Map<String, Libro> masLibros = new HashMap<,>();

. . .

. . .

libros.putAll(masLibros);
```

4.3. TreeSet

TreeSet es una colección que mantiene los elementos ordenados de forma natural o bien a través de un Comparator.

Operaciones más comunes

Añadir un elemento

```
Set<String> cadenas = new TreeSet<>();

String unaCadena = "Esto es una cadena";

cadenas.add(unaCadena);
```

Añadir todos los elementos de otra `Collection`

```
List<String> masCadenas = new ArrayList<>();
```

```
. . .

cadenas.addAll(masCadenas);
```

Obtener un elemento

Eliminar todos los elementos

```
cadenas.clear();
```

Comprueba si existe un elemento en la lista

```
String otraCadena = "Esto es una cadena";

if (cadenas.contains(otraCadena)) {

  System.out.println("La cadena existe");

}
```

Obtener el primer elemento

```
String primeraCadena = cadenas.first();
```

Obtener y eliminar el primer elemento

```
String primeraCadena = cadenas.pollFirst();
```

Obtener el último elemento

```
String ultimaCadena = cadenas.last();
```

Obtener y eliminar el último elemento

```
String ultimaCadena = cadenas.pollLast();
```

Comprobar si está vacío

```
if (cadenas.isEmpty()) {
```

```
  System.out.println("El Set de cadenas esta vacío");

}
```

Comprobar el número de elementos

```
System.out.println("Tienes " + cadenas.size() + " cadenas");
```

Obtiene la parte del `Set` cuyos elementos son menores que uno pasado por parámetro

```
String cadenaLimite = "Una cadena";

SortedSet<String> cadenasMenores = cadenas.headSet(cadenaLimite);

// SortedSet es el interface que implementa TreeSet
```

Obtiene la parte del `Set` cuyos elementos están entre dos determinados

```
String cadenaMenor = "Una cadena";

String cadenaMayor = "Más cadena";

SortedSet<String> cadenasMenores = cadenas.subSet(cadenaMenor,
cadenaMayor);

// SortedSet es el interface que implementa TreeSet
```

4.4. TreeMap

TreeMap es una colección de pares *clave-valor* como HashMap pero mantiene los elementos ordenados por su clave de forma natural o bien a través de un Comparator.

Operaciones más comunes

Añadir un elemento (pareja clave-valor)

```
Map<String, Libro> libros = new TreeMap<,>();

. . .

Libro libro = new Libro();
```

```
libro.setTitulo("Secuestrado");

libro.setAutor("Robert Louis Stevenson");

. . .

libros.put(libro.getTitulo(), libro);
```

Obtener un elemento

```
String tituloLibro = "Secuestrado";

Libro esteLibro = libros.get(titulo);
```

Comprobar si existe una clave

```
String titulo = "Secuestrado";

if (libros.containsKey(titulo)) {

  System.out.println("El libro con el título " + titulo + " existe en tu
colección");

}
```

Comprobar si está vacío

```
if (libros.isEmpty()) {

  System.out.println("Tu colección de libros está vacía");

}
```

Eliminar todos los elementos

```
libros.clear();
```

Eliminar un elemento (por clave)

```
String titulo = "Secuestrado";
```

```
libros.remove(titulo);
```

Obtener una `Collection` con todos los valores

```
Collection<Libro> coleccionLibros = libros.values();

// ArrayList, TreeSet y LinkedList implementan el interface Collection
```

Obtener un `Set` con todas las claves

```
Set<String> titulos = libros.keySet();

// TreeSet y HashSet implementan el interface Set
```

Comprobar el tamaño del `Map`

```
System.out.println("Tienes " + libros.size() + " libros en tu
colección");
```

Concatenar todos los elementos de otro `Map`

```
Map<String, Libro> masLibros = new TreeMap<,>();

. . .

. . .

libros.putAll(masLibros);
```

4.5. LinkedList

LinkedList Es una lista doblemente enlazada que implementa el interfaz List

Operaciones más comunes

Añadir un elemento

```
List<Libro> listaLibros = new LinkedList<>();

. . .
```

```
Libro libro = new Libro();

libro.setTitulo("Secuestrado");

libro.setAutor("Robert Louis Stevenson");

. . .

listaLibros.add(libro);
```

Añadir un elemento al principio

```
List<Libro> listaLibros = new LinkedList<>();

. . .

Libro libro = new Libro();

libro.setTitulo("Secuestrado");

libro.setAutor("Robert Louis Stevenson");

. . .

listaLibros.addFirst(libro);
```

Añadir un elemento al final

```
List<Libro> listaLibros = new LinkedList<>();

. . .

Libro libro = new Libro();

libro.setTitulo("Secuestrado");

libro.setAutor("Robert Louis Stevenson");

. . .

listaLibros.addLast(libro);
```

Añadir toda una colección al final

```
List<Libro> listaLibros = new LinkedList<>();

List<Libro> otraListaDeLibros = new ArrayList<>();

. . .

// Podemos añadir cualquier objeto que implemente el interface Collection

listaLibros.addAll(otraListaDeLibros);
```

Obtener un elemento

```
Libro unLibro = listaLibros.get(4);
```

Obtener el primer elemento

```
Libro unLibro = listaLibros.getFirst();
```

Eliminar (y obtener) el primer elemento

```
Libro primerLibro = listaLibros.removeFirst();
```

Eliminar (y obtener) el último elemento

```
Libro ultimoLibro = listaLibros.removeLast();
```

Eliminar (y obtener) un elemento

```
Libro unLibro = listaLibros.remove(3);
```

Eliminar todos los elementos

```
listaLibros.clear();
```

Obtener el número de elementos de la lista

```
System.out.println("Esta lista tiene " + listaLibros.size() + " libros");
```

Obtener un array (estático) con todos los elementos de la lista

```
Libro[] arrayLibros = listaLibros.toArray();
```

4.6. Comparativa de la JFC (Java Collection Framework)

JCF Class	Positioning	Ordering	Duplicates	Sample Use
TreeSet	None	Ascending	Not Allowed	Keep unique list of words from a document sorted
HashSet	None	None	Not Allowed	Keeps a unique list of words from a document to be referenced frequently
ArrayList	Numerical	None	Allowed	Keeps an "in-memory" lookup table of static data
LinkedList	Numerical	Insertion Order	Allowed	Keeps a list of words that change frequently
HashMap	Object as key	None	Unique Keys	Maps words to word frequency
TreeMap	Object as key	Ascending	Unique Keys	Maps sorted words to word frequency

JCF Classes Cheat Sheet

JCF: Java Collection Framework

5. Patrón Modelo-Vista-Controlador

El patrón Modelo-Vista-Controlador o MVC (Model-View-Controller en inglés) es un patrón de diseño que busca separar, en una aplicación con GUI, la lógica de la presentación de los datos facilitando así la reutilización de código y el mantenimiento de la aplicación.

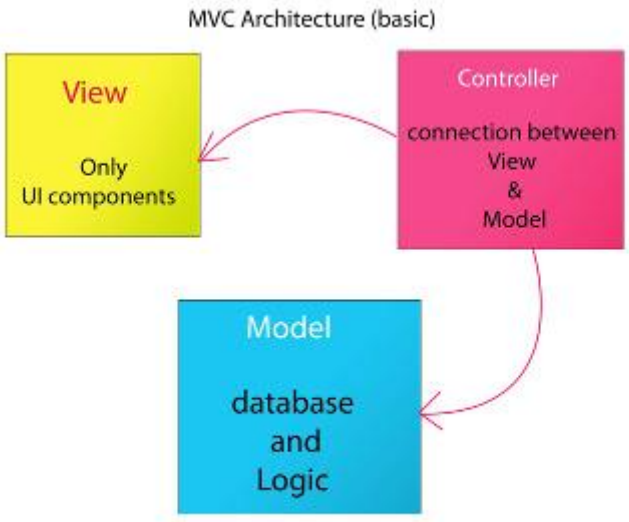


Figure 1: Model-View-Controller

5.1. Ejemplo de aplicación MVC

A continuación, se muestra el código completo de una pequeña aplicación con GUI de gestión con las operaciones más básicas (Alta, Modificar, Baja, Búsqueda) gestionada mediante colecciones Java.

Según el patrón MVC, la aplicación quedará separada en tres partes: *Model*, *View* y *Controller*. A esas 3 partes habrá que añadir las clases *POJOs (Plain Old Java Object)* que son las clases que representan a los objetos o elementos que se gestionan en la aplicación. Contendrán atributos, constructores, getters, setters y los métodos que proceda implementar para cada clase. En este caso hay una única clase *POJO* que es `Animal.java` puesto que la aplicación permite gestionar una base de datos de animales, en este caso mediante colecciones Java de forma que queda como ejercicio muy interesante transformar esta aplicación en una que gestione la información con Bases de Datos (MySQL ó PostgreSQL). Será entonces cuando se aprecie la separación Model-View-Controller y la reutilización real de código entre ambas versiones de esta aplicación.



Animal.java

```java
/**
 * Clase que representa a un Animal
 */
public class Animal {


  private String nombre;
  private String raza;
  private String caracteristicas;
  private float peso;


  // Constructores, getters y setters
  // Otros métodos (cuando proceda)
}
```

La clase `Animal` representa a los animales que vamos a gestionar y contendrá tantos atributos como características queramos almacenar del mismo. Además haremos los correspondientes constructores, *getters* y *setters*. Si procede, podemos añadir métodos que realicen operaciones sobre la clase.

Animal.java (Todo el código)

Ventana.java

```
/**
 * Clase que contiene la GUI de la Aplicación
 */
public class Ventana {
  private JPanel panel;
  JTextField tfNombre;
  JTextField tfRaza;
  // Resto de controles swing
  . . .

  public Ventana() {

    JFrame frame = new JFrame("Ventana");
    frame.setContentPane(panel);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.pack();
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
```

```
    }

}
```

La clase `Ventana` contendrá exclusivamente el diseño de la ventana y en el constructor la inicialización de la misma según convenga en cada caso. Ni siquiera los *Listeners* aparecerán en esta ventana. Por comodidad, no utilizaremos ningún modificador de accesibilidad para los componentes de la ventana, puesto que deberemos poder acceder a ellos desde el *Controller*. Otra manera será proporcionar *getters* para todos ellos y así podremos declararlos como privados o lo que mejor convenga.

Ventana.java (Todo el código)

VentanaController.java

```
/**
 * Controlador para la ventana
 * Recibe los inputs del usuario desde la ventana (View) y se
 * comunica con el Model si es necesario
 */
public class VentanaController implements ActionListener,
KeyListener {


  private VentanaModel model;
  private Ventana view;


  private int posicion;


  public VentanaController(VentanaModel model, Ventana view) {
    this.model = model;
    this.view = view;
```

```
    anadirActionListener(this);
    anadirKeyListener(this);

    posicion = 0;
  }

  @Override
  public void actionPerformed(ActionEvent event) {

    String actionCommand = event.getActionCommand();
    Animal animal = null;

    switch (actionCommand) {
      case "Nuevo":
        view.tfNombre.setText("");
        view.tfNombre.setEditable(true);
        view.tfCaracteristicas.setText("");
        view.tfCaracteristicas.setEditable(true);
        // Limpiar/Resetear resto de componentes
        . . .
        . . .
```

```
            view.btGuardar.setEnabled(true);

            break;

        case "Guardar":

            if (view.tfNombre.getText().equals("")) {

               Util.mensajeError("El nombre es un campo obligatorio",
"Nuevo Animal");

               return;

            }


            animal = new Animal();

            animal.setNombre(view.tfNombre.getText());

            animal.setRaza(view.tfRaza.getText());


animal.setCaracteristicas(view.tfCaracteristicas.getText());

            animal.setPeso(Float.parseFloat(view.tfPeso.getText()));


            model.guardar(animal);


            view.btGuardar.setEnabled(false);

            break;

        case "Modificar":

            animal = new Animal();

            animal.setNombre(view.tfNombre.getText());
```

```
            animal.setRaza(view.tfRaza.getText());

animal.setCaracteristicas(view.tfCaracteristicas.getText());
            animal.setPeso(Float.parseFloat(view.tfPeso.getText()));


            model.modificar(animal);
            break;
        case "Cancelar":
            view.tfNombre.setEditable(false);
            view.tfCaracteristicas.setEditable(false);
            . . .


            animal = model.getActual();
            cargar(animal);


            view.btGuardar.setEnabled(false);
            break;
        case "Eliminar":
            if (JOptionPane.showConfirmDialog(null, "¿Está seguro?",
"Eliminar",
              JOptionPane.YES_NO_OPTION) == JOptionPane.NO_OPTION)
              return;
```

```
            model.eliminar();

            animal = model.getActual();

            cargar(animal);

            break;

        case "Buscar":

            animal = model.buscar(view.tfBusqueda.getText());

            if (animal == null) {

              Util.mensajeInformacion("No se ha encontrado ningún
animal con ese nombre",

                "Buscar");

              return;

            }

            cargar(animal);

            break;

        . . .

        . . .

        default:

            break;

        }

    }



    . . .
```

```
. . .


@Override
public void keyReleased(KeyEvent e) {
  if (e.getSource() == view.tfBusqueda) {
    if (e.getKeyCode() == KeyEvent.VK_ENTER) {
      view.btBuscar.doClick();
    }
  }
}


/**
 * Carga los datos de un animal en la vista
 * @param animal
 */
private void cargar(Animal animal) {
  if (animal == null)
    return;


  view.tfNombre.setText(animal.getNombre());
  view.tfCaracteristicas.setText(animal.getCaracteristicas());
  . . .
```

```
        . . .
    }


    . . .
    /**
     * Añade el Listener a todos los controles de la View
     */
    private void anadirActionListener(ActionListener listener) {


        view.btNuevo.addActionListener(listener);
        view.btGuardar.addActionListener(listener);
        . . .
        . . .
    }
}
```

La clase `VentanaController` contiene el *Controller* de la ventana, que hará de intermediario entre la capa *View* que es la GUI de la aplicación y el *Model* que contiene toda la lógica y el acceso a los datos. Además, se encarga de gestionar los eventos de cada componente de la GUI (implementando los *Listeners* necesarios).

También se incluyen en este componente todas las validaciones de datos relacionadas con el *input* del usuario y la visualización de los mensajes de errores que correspondan.

VentanaController.java (Todo el código)

VentanaModel.java

```
/**
 * Modelo para la ventana
```

```
 */
public class VentanaModel {


  private ArrayList<Animal> listaAnimales;
  private int posicion;


  public VentanaModel() {
    listaAnimales = new ArrayList<>();
    posicion = 0;
  }


  /**
   * Guarda un animal en la lista
   * @param animal
   */
  public void guardar(Animal animal) {


    listaAnimales.add(animal);
    posicion++;
  }


  /**
```

```
    * Modifica los datos del animal actual
    * @param animalModificado
    */
   public void modificar(Animal animalModificado) {
     Animal animal = listaAnimales.get(posicion);
     animal.setNombre(animalModificado.getNombre());
animal.setCaracteristicas(animalModificado.getCaracteristicas());
     animal.setRaza(animalModificado.getRaza());
     animal.setPeso(animalModificado.getPeso());
   }

   /**
    * Elimina el animal actual
    */
   public void eliminar() {
     listaAnimales.remove(posicion);
   }

   public Animal getActual() {
     return listaAnimales.get(posicion);
   }
```

```java
/**
 * Busca un animal en la lista
 * @param nombre El nombre del animal
 * @return El animal o null si no se ha encontrado nada
 */
public Animal buscar(String nombre) {
  for (Animal animal : listaAnimales) {
    if (animal.getNombre().equals(nombre)) {
      return animal;
    }
  }

  return null;
}

/**
 * Obtiene el animal que está en primera posición en la lista
 * @return
 */
public Animal getPrimero() {
  posicion = 0;
```

```
    return listaAnimales.get(posicion);
  }


  /**
   * Obtiene el animal que está en la posición anterior a la actual
   * @return
   */
  public Animal getAnterior() {
    if (posicion == 0)
      return null;

    posicion--;
    return listaAnimales.get(posicion);
  }


  /**
   * Obtiene el animal que está en la posición siguiente a la
actual
   * @return
   */
  public Animal getSiguiente() {
```

```
        if (posicion == listaAnimales.size() - 1)
            return null;

        posicion++;
        return listaAnimales.get(posicion);
    }

    /**
     * Obtiene el animal que está en la última posición de la lista
     * @return
     */
    public Animal getUltimo() {

        posicion = listaAnimales.size() - 1;
        return listaAnimales.get(posicion);
    }
}
```

La clase `VentanaModel`, puesto que es el *Model*, contendrá todos los métodos que proporcionan el acceso a los datos y la lógica de la aplicación. En este caso contiene todos los métodos que permite obtener la información de los animales, los resultados de las búsquedas y también se llevan a cabo las operaciones de inserción, modificación y eliminación.

En este caso todas las operaciones se realizan sobre una colección `ArrayList`. Si quisiéramos hacer que la aplicación utilizara una Base de Datos como motor de almacenamiento sólo tendríamos que modificar este componente de la aplicación haciendo que los mismos métodos (incluso respetando su declaración: nombre, tipo devuelto y

parámetros) utilicen una Base de Datos para realizar todas las operaciones. No sería necesario realizar ninguna modificación en el resto de la aplicación.

VentanaModel (Todo el código)

Aplicacion.java

```
/**
 * Aplicación para la gestión de animales utilizando el patrón MVC
 *
 * Clase principal, que solamente lanza la aplicación
 */
public class Aplicacion {

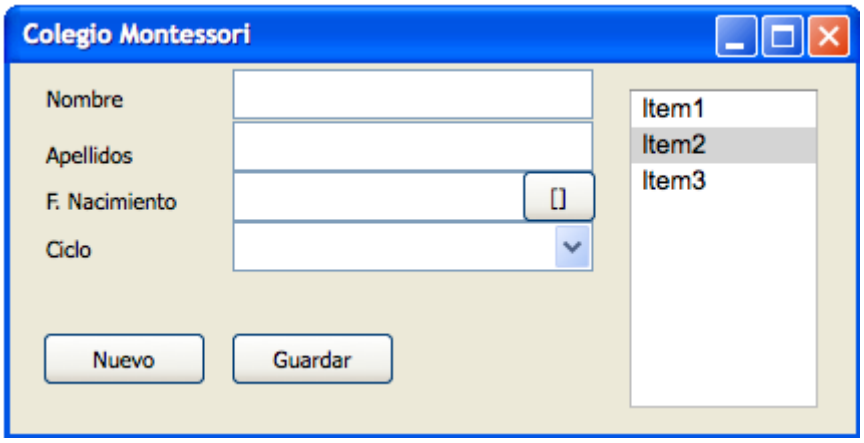

  public static void main(String args[]) {


    Ventana view = new Ventana();
    VentanaModel model = new VentanaModel();
    VentanaController controller = new VentanaController(model,
view);
  }
}
```

En la clase `Aplicacion`, que contiene el método `main` que lanzará la aplicación, simplemente tenemos que instanciar los tres componentes de nuestra aplicación (*Model*, *View* y *Controller*) y "conectarlos" mediante el último.

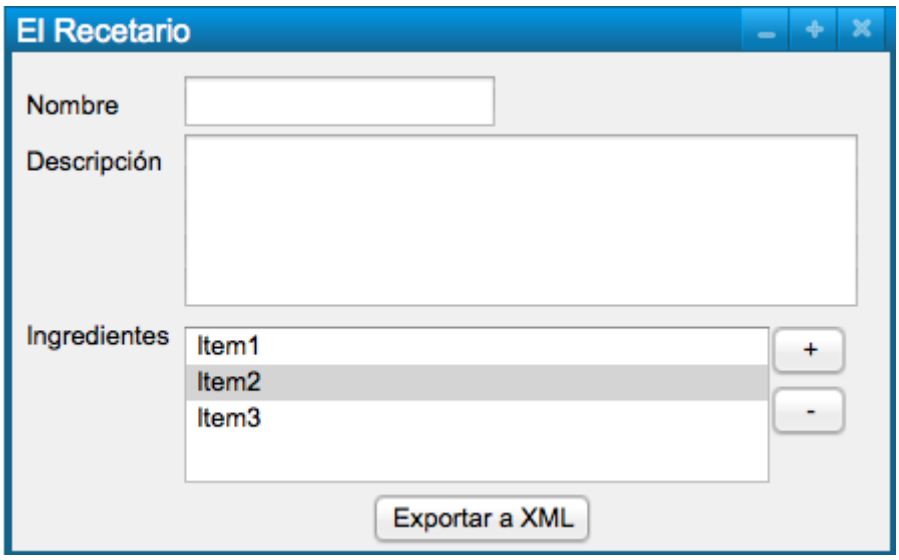
6. Ejercicios

1. Escribe un programa capaz de registrar información sobre los alumnos de un Colegio (nombre, apellidos, fecha de nacimiento y ciclo que estudian) y listarla en un JList. La información que se muestra en la lista se mantendrá actualizada en todo momento. Diseña la aplicación según el patrón de diseño MVC (Model-View-Controller)

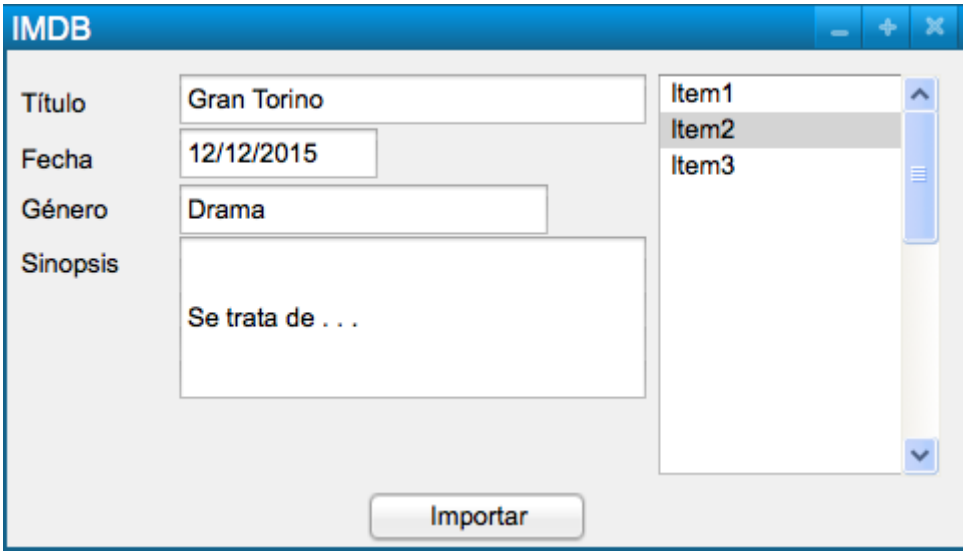


2. Añade al programa anterior los botones necesarios para poder "navegar" (ir al primero, anterior, siguiente y último) por los registros de la lista
3. Añadir un botón al primer ejercicio para que el usuario guarde la información de la lista en un fichero. El usuario tendrá que escoger cada vez el fichero donde quiere almacenar la información

4. Crea una aplicación que sea capaz de exportar a XML la información de una receta (nombre, descripción, ingredientes)



5. Crea una aplicación que sea capaz de importar un fichero XML con información de un catálogo de películas (titulo, fecha, genero, sinopsis y actores principales). Diseña la GUI para que se pueda visualizar toda la información de cada una de las películas importadas



6. Crea una aplicación que sirva de Bloc de Notas con las funciones de *Nuevo*, *Guardar* y *Guardar como*. Diseña la aplicación según el patrón de diseño MVC (Model-View-Controller)

7. Práctica 1

1. Realiza una aplicación que almacene en un fichero una lista de objetos `Ordenador` (modelo, marca, precio y fecha de compra).

2. Implementa una aplicación que contenga un `JComboBox` que sea capaz de listar el contenido de una lista de objetos `Ordenador` (modelo, marca, precio y fecha de compra), mostrando para cada uno de ellos el nombre y su precio.

3. Escribe una aplicación con interfaz gráfico que sea capaz de almacenar una lista de objetos `Persona` (nombre, apellidos, fecha de nacimiento y salario) en un fichero. La aplicación tendrá que funcionar siempre mostrando errores cuando se introduzcan valores incorrectos. El usuario tendrá que ser capaz de:
    a. Registrar personas
    b. Exportar toda la información almacenada a un fichero XML
    c. Listar las personas registradas

4. Escribe una aplicación que añada cadenas de texto a un JList y que contenga un botón que almacene todas ellas a un fichero.

5. Implementa una aplicación que contenga un `JList` que sea capaz de listar el contenido de una lista de objetos `Ordenador` (con modelo, marca, precio y fecha de compra) mostrando para cada uno de ellos el nombre y su precio.

6. Escribe una aplicación que contenga un JComboBox que liste cadenas de texto y que contenga un botón que almacene todas ellas en un fichero.

7. Realiza una aplicación que almacene en un fichero la información de un objeto `Persona` (nombre, apellidos, salario y fecha de nacimiento).

8. Realiza una aplicación para registrar un número indeterminado de personas (almacenando de cada una el nombre, apellidos, salario y fecha de nacimiento), permitiendo también la opción de listarlas mostrando nombre y fecha de nacimiento.

9. Realiza una aplicación que permita añadir cadenas de texto a un `JList` y que permita almacenarlas a un fichero que el usuario escogerá.

10. Realiza una aplicación que almacene en un fichero una serie de datos introducidos por el usuario (nombre, apellidos, fecha de nacimiento, dirección postal y email).

11. Realiza una aplicación que contenga un `JComboBox` que sea capaz de listar una lista de cadenas de texto que están almacenadas en un fichero.

12. Escribe una aplicación que importe un fichero XML cuyo formato se indica a continuación:

enemigos.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="no">
```

```
<xml>
  <pantalla>
    <enemigo>
      <nombre>superman</nombre>
      <ataque>15</ataque>
      <vida>100</vida>
    </enemigo>
    <enemigo>
      <nombre>batman</nombre>
      <ataque>150</ataque>
      <vida>200</vida>
    </enemigo>
    <enemigo>
      <nombre>spiderman</nombre>
      <ataque>5</ataque>
      <vida>10</vida>
    </enemigo>
  </pantall>
</xml>
```

8. Práctica 2

Objetivos

Desarrollar una aplicación que almacene datos en un fichero

Enunciado

Se debe implementar una aplicación que gestione información almacenándola y recuperándola de fichero. Se pensará en un supuesto real en el que exista un tipo de objeto y se creará la aplicación para gestionarlo.

Requisitos (1 pto cada uno)

- Se debe gestionar información de un tipo de objeto con, al menos, 5 atributos. Deben aparecer, al menos, datos de tipo cadena, número (entero y decimal) e imágenes.
- De cada objeto el usuario podrá dar de alta, modificar y eliminar
- La aplicación será capaz de almacenar toda la información en disco en una ubicación fija de forma transparente para el usuario. La carga de los datos se realizará durante la carga de la aplicación
- Se deberá comentar el código
- La aplicación contará con un listado para cada tipo de objeto de forma que el usuario pueda acceder a cualquiera de ellos directamente para ver su información

Otras funcionalidades (1 pto cada una)

- La aplicación contará con una opción de búsqueda avanzada desde la que se podrá buscar algún objeto
- La aplicación dispondrá de una opción de *guardar como* que permitirá almacenar los datos en una ubicación alternativa
- Añadir la opción de que sea el usuario quién decida cuando realizar el guardado de los datos, en lugar de hacerlo de forma transparente
- Permitir al usuario cambiar, desde la aplicación, la ruta fija donde se almacenan los ficheros
- Añadir una opción a la aplicación que permita eliminar todos los datos del programa
- Activar/Desactivar los controles del interfaz de forma que no se permita al usuario utilizar aquellos que no deba usar en cada momento
- Añadir una opción al usuario que permita recuperar el último elemento borrado